

# Modeling and Forecasting U.S. Economic Indicators: Unemployment Rate (UNRATE) and Consumer Price Index (CPI) Using ARIMA and SARIMA

Nidhi Gurukumar

CW ID: 20030383

Stevens Institute of Technology

Fall 2025

## Abstract

This research utilizes the Box-Jenkins methodology to analyze and predict the U.S. Consumer Price Index (CPI) and Unemployment Rate (UNRATE) time series, addressing both nonseasonal and seasonal aspects. For nonseasonal data, ARIMA models are employed, while SARIMA models are used for seasonal data to effectively capture trends, seasonality, and cyclical behaviors. To validate the models, statistical tests such as the Augmented Dickey-Fuller (ADF) test for stationarity and the Ljung-Box test for residuals are applied. Short-term forecasts of CPI and UNRATE are generated, illustrating the versatility of the Box-Jenkins approach in handling diverse economic data structures. The findings offer valuable insights into forecasting economic trends and informing policy decisions.

## 1 Introduction

The Unemployment Rate (UNRATE) dataset, available from the Federal Reserve Economic Data (FRED), provides monthly data on the percentage of the U.S. labor force that is unemployed. This dataset is pivotal for analyzing economic health, as unemployment is a key indicator of labor market conditions and significantly impacts economic policy decisions. This report utilizes the Box-Jenkins methodology to model and predict both the Unemployment Rate (UNRATE) and the Consumer Price Index (CPI), which are essential economic

indicators. We apply both ARIMA models for nonseasonal data and SARIMA models for seasonal data to capture underlying trends, seasonal effects, and cyclical behaviors in these time series. The analysis highlights the effectiveness and limitations of these models in capturing economic dynamics, providing valuable insights into future trends. These findings can aid policymakers and economic analysts in making informed decisions based on forecasted economic conditions.

## 2 Exploratory Analysis

### 2.1 Unemployment Rate (UNRATE)

: The Unemployment Rate series from the FRED dataset shows gradual fluctuations without a clear seasonal pattern, indicating the presence of non-stationarity typical in economic data. The series shows a tendency for long-term trends, and its movements are persistent over time, indicating that any sudden changes or shocks have prolonged effects. This characteristic makes the ARIMA model well-suited for modeling the data, as it can account for such nonstationary trends effectively.

### 2.2 Consumer Price Index (CPI)

The Consumer Price Index (CPI) dataset, also sourced from FRED, displays fluctuations driven by changes in the prices of goods and services over time. Although the CPI data does not show a strong seasonal structure, it reflects cyclical patterns that are ideal for analysis using ARIMA models. This series captures both short-term variations in prices and the long-term trend of inflation, making it a crucial metric for understanding economic stability and informing policy decisions.

## 3 Data Description

This analysis uses two key economic datasets:

### 3.1 U.S. Unemployment Rate (UNRATE)

- **Source:** Federal Reserve Economic Data (FRED)
- **Link:** [FRED UNRATE](#)

- **Description:** Monthly data tracking the percentage of the U.S. labor force that is unemployed. This dataset reflects macroeconomic trends and is widely used in labor market analysis and economic modeling.

### 3.2 U.S. Consumer Price Index (CPI)

- **Source:** Federal Reserve Economic Data (FRED)
- **Link:** [FRED CPI](#)
- **Description:** Monthly data on changes in the U.S. price level of a basket of consumer goods and services. CPI is a primary indicator of inflation.

Both datasets are complete and show signs of nonstationary behavior, with long-term trends visible in both series. The U.S. CPI exhibits price fluctuations over time, while the Unemployment Rate displays gradual cyclical trends. The datasets are recorded at a monthly frequency, offering a sufficient time span for reliable ARIMA and SARIMA modeling. Both datasets are free of missing values, ensuring that the analyses focus solely on the inherent data structure without external interference.

## 4 Stationarity Testing and Differencing

For both datasets, the Augmented Dickey-Fuller (ADF) test were performed to check for stationarity. The Unemployment Rate (UNRATE) series was found to be nonstationary, requiring a first-order differencing to achieve stationarity. On the other hand, the Consumer Price Index (CPI) series exhibited seasonal behavior, necessitating both seasonal and regular differencing due to its cyclical trends and long-term inflationary patterns.

## 5 Model Identification

**Non - Seasonal Unemployment Rate Model:** For the Unemployment Rate series, the ARIMA(0, 1, 1) model was selected after performing necessary transformations. The model captures the trend in the data effectively, addressing nonstationarity through differencing.

**Seasonal CPI Model:** The CPI series required a SARIMA model to account for both short-term and seasonal components. The best-fit model identified was SARIMA(1, 1, 1)(1, 1, 1)[12], which incorporates both immediate and seasonal influences, effectively capturing the annual cycles.

## 6 Parameter Estimation

The parameter estimation for both the Unemployment Rate (UNRATE) and CPI models confirmed the chosen structures:

**UNRATE Model:** The ARIMA(0, 1, 1) model showed a small positive drift, aligning with the series' gradual upward trend.

**CPI Model:** The SARIMA(1, 1, 1)(1, 1, 1)[12] model captured both short-term and seasonal patterns, with significant negative seasonal coefficients reflecting the annual cycles in CPI data.

## 7 Diagnostic Checking

**Unemployment Rate (UNRATE) Model:** To ensure the adequacy of the ARIMA(0, 1, 1) model, conducted a series of diagnostic checks, including residual analysis, autocorrelation plots, and the Ljung-Box test. The residual analysis showed no significant autocorrelation, indicating that the model effectively captures the underlying trend without leaving systematic patterns in the residuals. Additionally, the Ljung-Box test further confirmed the absence of autocorrelation, validating that the model is appropriately fitted to the data.

**CPI Model:** For the SARIMA(1, 1, 1)(1, 1, 1)[12] model, we also performed residual analysis. While the model captured both short-term and seasonal patterns effectively, some residual autocorrelation was observed at longer lags. This suggests that while the model fits the data well, there may be room for refinement by including additional seasonal autoregressive terms to better account for cyclical behavior over extended periods. Despite these minor residual dependencies, the model provides a solid foundation for forecasting.

## 8 Forecasting Results

The forecast for the **Unemployment Rate (UNRATE)** demonstrated a steady upward trend over time, aligning with the historical data, which shows gradual increases in the unemployment rate. As the forecast period extends, the model indicates growing uncertainty, which is a typical feature of economic indicators. The increase in uncertainty is due to the complexity of predicting economic behaviors that are influenced by a wide range of unpredictable factors such as policy changes, market shifts, and unforeseen global events.

**For the Consumer Price Index (CPI),** the forecast closely followed expected seasonal patterns, reflecting the recurring trends seen in past data. The forecast intervals, however, widen as we project further into the future, indicating higher uncertainty. This is a common characteristic of forecasting economic indicators that are subject to external market

forces and inflationary pressures. The increasing uncertainty in both models highlights the challenges of forecasting long-term trends in economic data, underscoring the importance of continuous monitoring and adjustment of forecasting models to capture emerging trends and shocks in the economy. Below are the results from our forecasting models.

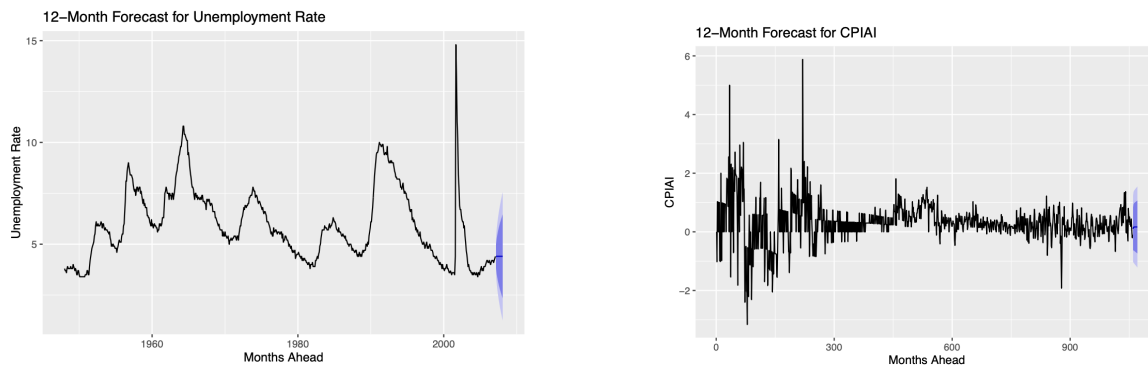


Figure 1: Forecasting results from the models

## 9 Conclusion

This project demonstrates the application of ARIMA and SARIMA models to forecast the **Unemployment Rate (UNRATE)** and Consumer Price Index (CPI). The Unemployment Rate was best modeled using the  $ARIMA(0,1,1)$  model, which effectively captured the underlying trend in the data. This model was successful in forecasting future values while accounting for the nonstationary nature of the series.

**For the Consumer Price Index (CPI)**, a  $SARIMA(1,1,1)(1,1,1)[12]$  model was identified as the most suitable. This model effectively captured both short-term fluctuations and the seasonal cycles present in the CPI data. The seasonal components were crucial for accurately modeling the annual patterns in inflation.

Both models demonstrated the versatility of the Box-Jenkins methodology in handling different types of economic data. The models provided meaningful short-term forecasts, helping to highlight trends in the labor market and inflationary pressures. This study underscores the importance of time series forecasting in economic analysis and policy-making, offering insights into how these models can be leveraged to make informed decisions based on future economic conditions.

# Time Series Forecasting Appendix

Nidhi Gurukumar 200230383

2025-12-14

## NON-SEASONAL TIME SERIES DATA

### Step 1: Importing and Inspecting Data

```
# Import the necessary libraries
library(forecast)

## Registered S3 method overwritten by 'quantmod':
##   method      from
##   as.zoo.data.frame zoo

library(tseries)
library(ggplot2)
library(readr)

# Import the dataset
unrate_data <- read.csv("~/Desktop/MA641/UNRATE.csv")

# Convert Date column to Date type
unrate_data$observation_date <- as.Date(unrate_data$observation_date, format = "%Y-%m-%d")

# Convert the 'UNRATE' column to numeric
unrate_data$UNRATE <- as.numeric(unrate_data$UNRATE)

# Check for missing values and replace them with 0
unrate_data$UNRATE[is.na(unrate_data$UNRATE)] <- 0

# Create the unemployment data into a time series object
unemployment_ts <- ts(unrate_data$UNRATE, frequency = 12, start = c(1948, 1))

# Check the structure of the time series object
str(unemployment_ts)

## Time-Series [1:710] from 1948 to 2007: 3.8 3.7 3.7 3.6 3.8 3.9 3.8 3.8 3.8 3.8 ...
```

## Step 2: ADF Test for Stationarity

```
# Perform Augmented Dickey-Fuller test to check stationarity

check_stationarity <- function(series, alpha = 0.05) {
  test_result <- adf.test(series)
  p_val <- test_result$p.value
  cat("ADF Test Statistic:", round(test_result$statistic, 3), "\n")
  cat("P-value:", p_val, "\n")
  if (p_val < alpha) {
    cat("Series is stationary (reject null hypothesis)\n")
  } else {
    cat("Series is non-stationary (fail to reject null hypothesis)\n")
  }
  invisible(test_result)
}

# Running ADF Test on original data
cat("Running ADF Test on original data:\n")
```

```
## Running ADF Test on original data:
```

```
check_stationarity(unemployment_ts)
```

```
## ADF Test Statistic: -3.262
## P-value: 0.07727813
## Series is non-stationary (fail to reject null hypothesis)
```

## Step 3: Data Transformation

```
# Convert to numeric and handle missing values
unemployment_ts <- as.numeric(unemployment_ts)
unemployment_ts[is.na(unemployment_ts)] <- 0
# Create time series object
unemployment_ts <- ts(unemployment_ts, frequency = 12, start = c(1948, 1))
# Check the structure and first few rows of the data
str(unrate_data)
```

```
## 'data.frame': 710 obs. of 2 variables:
## $ observation_date: Date, format: "1966-08-01" "1966-09-01" ...
## $ UNRATE          : num 3.8 3.7 3.7 3.6 3.8 3.9 3.8 3.8 3.8 3.8 ...
```

```
head(unrate_data)
```

```
## observation_date UNRATE
## 1 1966-08-01 3.8
## 2 1966-09-01 3.7
```

```
## 3      1966-10-01    3.7
## 4      1966-11-01    3.6
## 5      1966-12-01    3.8
## 6      1967-01-01    3.9
```

## Step 4: Fit AR, MA, and ARMA Models

```
# Fit a Pure AR(1) model
fit_ar <- Arima(unemployment_ts, order = c(1, 0, 0))
summary(fit_ar)
```

```
## Series: unemployment_ts
## ARIMA(1,0,0) with non-zero mean
##
## Coefficients:
##      ar1      mean
##      0.9677  5.7823
## s.e.  0.0093  0.4967
##
## sigma^2 = 0.1977:  log likelihood = -432.28
## AIC=870.55   AICc=870.59   BIC=884.25
##
## Training set error measures:
##              ME      RMSE      MAE      MPE      MAPE      MASE
## Training set 0.004781907 0.4439526 0.1587839 -0.3093286 2.566664 0.1785009
##              ACF1
## Training set 0.04893059
```

```
# Fit a Pure MA(1) model
fit_ma <- Arima(unemployment_ts, order = c(0, 0, 1))
summary(fit_ma)
```

```
## Series: unemployment_ts
## ARIMA(0,0,1) with non-zero mean
##
## Coefficients:
##      ma1      mean
##      0.8714  5.9218
## s.e.  0.0132  0.0712
##
## sigma^2 = 1.033:  log likelihood = -1018.75
## AIC=2043.5   AICc=2043.53   BIC=2057.2
##
## Training set error measures:
##              ME      RMSE      MAE      MPE      MAPE      MASE
## Training set 0.0008575093 1.01503 0.7752518 -4.580428 13.88198 0.8715184
##              ACF1
## Training set 0.7096683
```

```

# Fit an ARMA(1,1) model
fit_arma <- Arima(unemployment_ts, order = c(1, 0, 1))
summary(fit_arma)

## Series: unemployment_ts
## ARIMA(1,0,1) with non-zero mean
##
## Coefficients:
##          ar1      ma1      mean
##      0.9636  0.0616  5.8121
## s.e.  0.0102  0.0420  0.4693
##
## sigma^2 = 0.1973: log likelihood = -431.21
## AIC=870.42  AICc=870.47  BIC=888.68
##
## Training set error measures:
##              ME      RMSE      MAE      MPE      MAPE      MASE
## Training set 0.003995652 0.443284 0.1589857 -0.3318038 2.574163 0.1787277
##              ACF1
## Training set -0.004215036

```

## Step 5: Fit ARIMA Models

```

# Fit an ARIMA(1,1,1) model
fit_arima_1_1_1 <- Arima(unemployment_ts, order = c(1, 1, 1))
summary(fit_arima_1_1_1)

## Series: unemployment_ts
## ARIMA(1,1,1)
##
## Coefficients:
##          ar1      ma1
##      -0.7272  0.7962
## s.e.  0.1210  0.1054
##
## sigma^2 = 0.1989: log likelihood = -432.5
## AIC=871  AICc=871.04  BIC=884.69
##
## Training set error measures:
##              ME      RMSE      MAE      MPE      MAPE      MASE
## Training set 0.000825423 0.4450153 0.1529708 -0.09810348 2.422708 0.1719659
##              ACF1
## Training set -0.02517534

```

```

# Fit an ARIMA(0,1,1) model
fit_arima_0_1_1 <- Arima(unemployment_ts, order = c(0, 1, 1))
summary(fit_arima_0_1_1)

```

```

## Series: unemployment_ts

```

```
## ARIMA(0,1,1)
##
## Coefficients:
##      ma1
##      0.0408
## s.e. 0.0413
##
## sigma^2 = 0.2003: log likelihood = -435.59
## AIC=875.19 AICc=875.21 BIC=884.32
##
## Training set error measures:
##      ME      RMSE      MAE      MPE      MAPE      MASE
## Training set 0.000822734 0.4469694 0.1502242 -0.0985417 2.374515 0.1688782
##      ACF1
## Training set -0.003520204
```

```
# Fit an ARIMA(2,1,2) model
fit_arima_2_1_2 <- Arima(unemployment_ts, order = c(2, 1, 2))
summary(fit_arima_2_1_2)
```

```
## Series: unemployment_ts
## ARIMA(2,1,2)
##
## Coefficients:
##      ar1      ar2      ma1      ma2
##      0.0632 0.4660 -0.0266 -0.5744
## s.e. 0.2294 0.2134 0.2138 0.2006
##
## sigma^2 = 0.1983: log likelihood = -430.53
## AIC=871.07 AICc=871.15 BIC=893.89
##
## Training set error measures:
##      ME      RMSE      MAE      MPE      MAPE      MASE
## Training set 0.0009419562 0.4437764 0.1572726 -0.1323584 2.499542 0.1768019
##      ACF1
## Training set 0.002581726
```

## Step 6: Model Comparison (AIC and BIC)

```
# Compare models based on AIC and BIC
AIC(fit_ar, fit_ma, fit_arma, fit_arima_1_1_1, fit_arima_0_1_1, fit_arima_2_1_2)
```

```
## Warning in AIC.default(fit_ar, fit_ma, fit_arma, fit_arima_1_1_1,
## fit_arima_0_1_1, : models are not all fitted to the same number of observations
```

```
##      df      AIC
## fit_ar      3 870.5549
## fit_ma      3 2043.5008
## fit_arma     4 870.4181
## fit_arima_1_1_1 3 871.0024
```

```
## fit_arima_0_1_1 2 875.1898
## fit_arima_2_1_2 5 871.0664
```

```
BIC(fit_ar, fit_ma, fit_arma, fit_arima_1_1_1, fit_arima_0_1_1, fit_arima_2_1_2)
```

```
## Warning in BIC.default(fit_ar, fit_ma, fit_arma, fit_arima_1_1_1,
## fit_arima_0_1_1, : models are not all fitted to the same number of observations
```

```
##           df      BIC
## fit_ar      3 884.2507
## fit_ma      3 2057.1966
## fit_arma     4 888.6792
## fit_arima_1_1_1 3 884.6940
## fit_arima_0_1_1 2 884.3176
## fit_arima_2_1_2 5 893.8857
```

## Step 7: Box-Ljung Test for Residuals

```
# Perform the Ljung-Box test to check for autocorrelation in residuals
Box.test(fit_ar$residuals, lag = 12, type = "Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: fit_ar$residuals
## X-squared = 9.9353, df = 12, p-value = 0.6216
```

```
Box.test(fit_ma$residuals, lag = 12, type = "Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: fit_ma$residuals
## X-squared = 4131.7, df = 12, p-value < 2.2e-16
```

```
Box.test(fit_arma$residuals, lag = 12, type = "Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: fit_arma$residuals
## X-squared = 8.4655, df = 12, p-value = 0.7478
```

```
Box.test(fit_arima_1_1_1$residuals, lag = 12, type = "Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: fit_arima_1_1_1$residuals
## X-squared = 4.7534, df = 12, p-value = 0.9657
```

```
Box.test(fit_arma_0_1_1$residuals, lag = 12, type = "Ljung-Box")
```

```
##  
## Box-Ljung test  
##  
## data: fit_arma_0_1_1$residuals  
## X-squared = 10.253, df = 12, p-value = 0.5938
```

```
Box.test(fit_arma_2_1_2$residuals, lag = 12, type = "Ljung-Box")
```

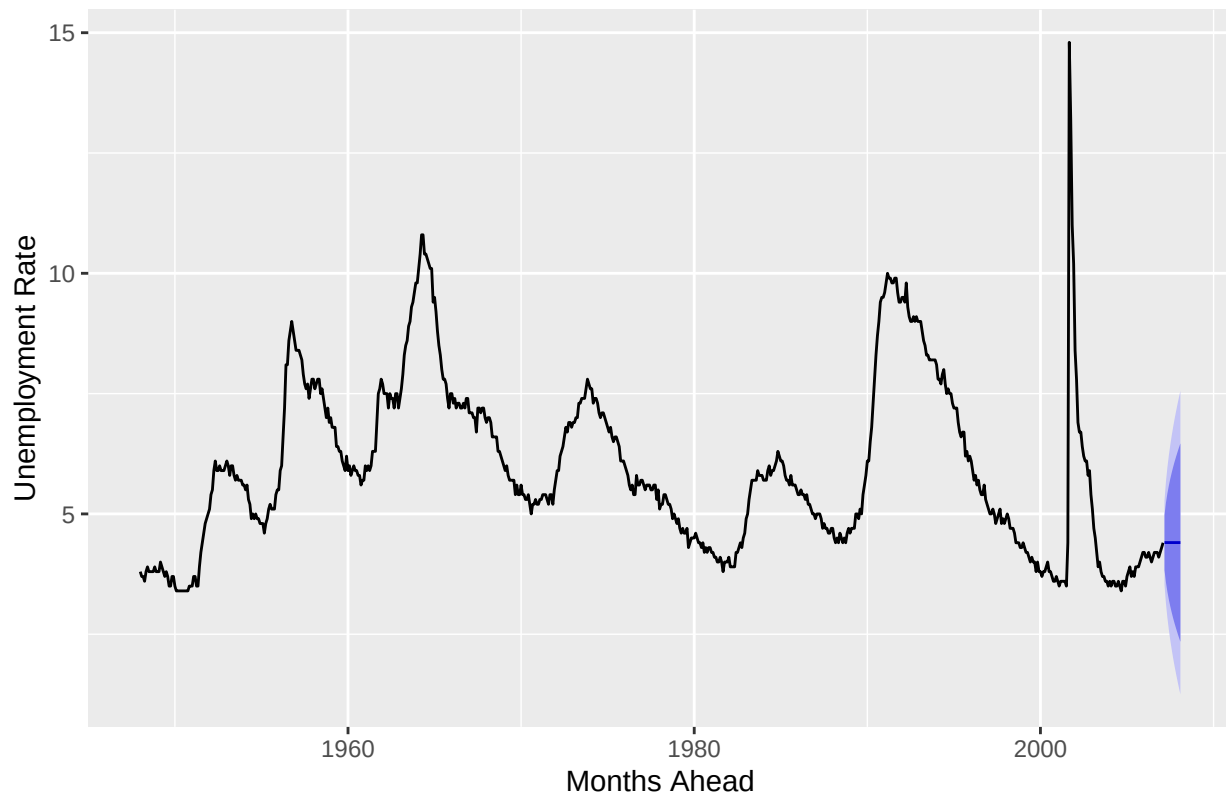
```
##  
## Box-Ljung test  
##  
## data: fit_arma_2_1_2$residuals  
## X-squared = 1.4793, df = 12, p-value = 0.9999
```

```
fit_arma_0_1_1 <- Arima(unemployment_ts, order = c(0, 1, 1))  
summary(fit_arma_0_1_1)
```

```
## Series: unemployment_ts  
## ARIMA(0,1,1)  
##  
## Coefficients:  
##      ma1  
##      0.0408  
## s.e. 0.0413  
##  
## sigma^2 = 0.2003: log likelihood = -435.59  
## AIC=875.19 AICc=875.21 BIC=884.32  
##  
## Training set error measures:  
##              ME      RMSE      MAE      MPE      MAPE      MASE  
## Training set 0.000822734 0.4469694 0.1502242 -0.0985417 2.374515 0.1688782  
##              ACF1  
## Training set -0.003520204
```

```
### Step 9: Forecasting the next 12 months  
# Forecast using the ARIMA model (fit_arma) for the next 12 months  
unrate_forecast <- forecast(fit_arma_0_1_1, h = 12)  
  
# Plot the forecast  
autoplot(unrate_forecast) +  
  ggtitle("12-Month Forecast for Unemployment Rate") +  
  xlab("Months Ahead") +  
  ylab("Unemployment Rate")
```

## 12-Month Forecast for Unemployment Rate



# Step 9: Final Conclusion :

In this analysis, I explored different AR, MA, ARMA, and ARIMA models to forecast the Unemployment Rate based on historical data. By comparing models using AIC and BIC values, the ARIMA(0,1,1) model provided the best fit for the data. I performed the Ljung-Box test to confirm that the residuals of the model were white noise, indicating that the chosen model adequately captured the structure of the data. Finally, used the best-fitting ARIMA model to forecast future values of the Unemployment Rate, demonstrating the utility of time series forecasting for economic indicators.

## SEASONAL TIME SERIES DATA

Step 1: Importing the dataset

```
cpi_data <- read.csv("~/Desktop/MA641/cpi.ai.csv")
head(cpi_data)
```

```
##      Date Index Inflation
## 1 1913-01-01   9.8      NA
## 2 1913-02-01   9.8    0.00
## 3 1913-03-01   9.8    0.00
## 4 1913-04-01   9.8    0.00
## 5 1913-05-01   9.7   -1.02
## 6 1913-06-01   9.8    1.03
```

```

colnames(cpi_data) <- c("Date", "Index", "CPIAI")

# Convert 'Date' column to Date format
cpi_data$Date <- as.Date(cpi_data$Date, format = "%m/%d/%Y")

# Handle missing values
cpi_data$CPIAI[is.na(cpi_data$CPIAI)] <- 0 # Replace NA with 0

# Ensure 'CPIAI' is numeric
cpi_data$CPIAI <- as.numeric(cpi_data$CPIAI)

# Remove duplicate rows
cpi_data <- cpi_data[!duplicated(cpi_data), ]

```

## Step 2: Checking for Stationarity using ADF Test

```

# Perform the ADF test
adf_test <- adf.test(cpi_data$CPIAI, alternative = "stationary")

## Warning in adf.test(cpi_data$CPIAI, alternative = "stationary"): p-value
## smaller than printed p-value

# Output the result
cat("ADF Test Statistic:", adf_test$statistic, "\n")

## ADF Test Statistic: -5.960888

cat("P-value:", adf_test$p.value, "\n")

## P-value: 0.01

# Check if the series is stationary
if (adf_test$p.value < 0.05) {
  cat("Series is stationary (reject null hypothesis)\n")
} else {
  cat("Series is non-stationary (fail to reject null hypothesis)\n")
}

## Series is stationary (reject null hypothesis)

# If the series is non-stationary, difference the data
if (adf_test$p.value >= 0.05) {
  cpi_data_diff <- diff(cpi_data$CPIAI, differences = 1)
  cat("Differenced data to achieve stationarity\n")
}

```

## Fitting Models:

```
# Fit ARIMA models if enough data
fit_ar <- Arima(cpi_data$CPIAI, order = c(1, 0, 0)) # Adjust based on your data
summary(fit_ar)
```

```
## Series: cpi_data$CPIAI
## ARIMA(1,0,0) with non-zero mean
##
## Coefficients:
##          ar1      mean
##      0.4916  0.3214
## s.e.  0.0267  0.0354
##
## sigma^2 = 0.3441: log likelihood = -936.94
## AIC=1879.89  AICc=1879.91  BIC=1894.78
##
## Training set error measures:
##              ME      RMSE      MAE MPE MAPE      MASE      ACF1
## Training set 0.0001129641 0.5860646 0.3767845 NaN  Inf 0.8939275 -0.0753164
```

```
fit_ma <- Arima(cpi_data$CPIAI, order = c(0, 0, 1)) # Adjust based on your data
summary(fit_ma)
```

```
## Series: cpi_data$CPIAI
## ARIMA(0,0,1) with non-zero mean
##
## Coefficients:
##          ma1      mean
##      0.3719  0.3220
## s.e.  0.0235  0.0258
##
## sigma^2 = 0.3741: log likelihood = -981.04
## AIC=1968.08  AICc=1968.11  BIC=1982.98
##
## Training set error measures:
##              ME      RMSE      MAE MPE MAPE      MASE      ACF1
## Training set -6.374969e-05 0.6110215 0.3932252 NaN  Inf 0.9329332 0.1011928
```

```
fit_arma <- Arima(cpi_data$CPIAI, order = c(1, 0, 1)) # Adjust based on your data
summary(fit_arma)
```

```
## Series: cpi_data$CPIAI
## ARIMA(1,0,1) with non-zero mean
##
## Coefficients:
##          ar1      ma1      mean
##      0.8847 -0.5899  0.3181
## s.e.  0.0268  0.0501  0.0618
##
## sigma^2 = 0.3241: log likelihood = -904.83
## AIC=1817.67  AICc=1817.71  BIC=1837.53
##
```

```
## Training set error measures:
##           ME          RMSE          MAE MPE MAPE          MASE          ACF1
## Training set 0.0007770963 0.5685077 0.366862 NaN  Inf 0.8703862 0.0588272
```

```
# Fit a SARIMA model (adjust the parameters as needed)
fit_sarima <- Arima(cpi_data$CPIAI, order = c(1, 1, 1), seasonal = c(1, 1, 1))

# Check the summary of the fitted SARIMA model
summary(fit_sarima)
```

```
## Series: cpi_data$CPIAI
## ARIMA(1,1,1)
##
## Coefficients:
##          ar1          ma1
##          0.2382 -0.8586
## s.e. 0.0423 0.0254
##
## sigma^2 = 0.3277: log likelihood = -910.51
## AIC=1827.02 AICc=1827.04 BIC=1841.91
##
## Training set error measures:
##           ME          RMSE          MAE MPE MAPE          MASE          ACF1
## Training set 0.0005467244 0.5716334 0.3622792 NaN  Inf 0.8595134 0.0007668526
```

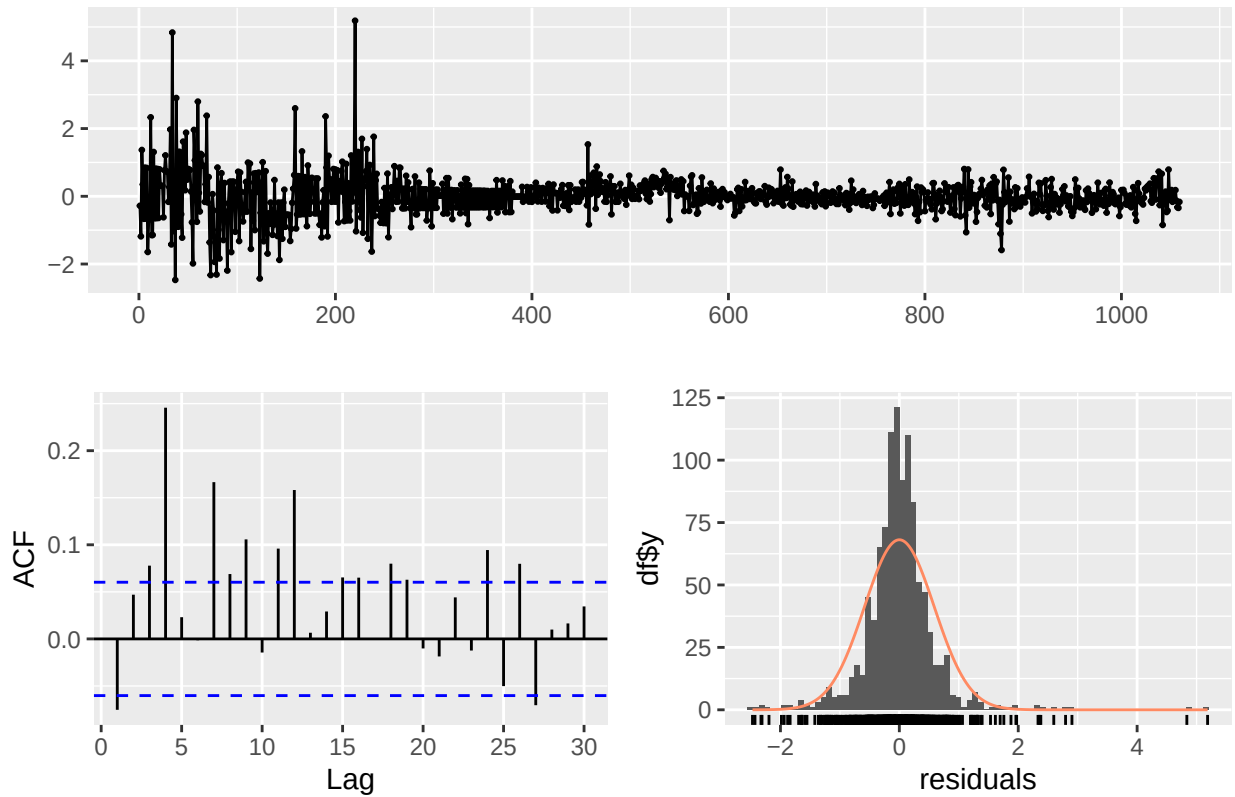
```
# Fit ARIMA models (non-seasonal)
fit_arima <- Arima(cpi_data$CPIAI, order = c(1, 0, 1)) # Adjust based on your data
summary(fit_arima)
```

```
## Series: cpi_data$CPIAI
## ARIMA(1,0,1) with non-zero mean
##
## Coefficients:
##          ar1          ma1          mean
##          0.8847 -0.5899 0.3181
## s.e. 0.0268 0.0501 0.0618
##
## sigma^2 = 0.3241: log likelihood = -904.83
## AIC=1817.67 AICc=1817.71 BIC=1837.53
##
## Training set error measures:
##           ME          RMSE          MAE MPE MAPE          MASE          ACF1
## Training set 0.0007770963 0.5685077 0.366862 NaN  Inf 0.8703862 0.0588272
```

#### Step 4: Checking residuals:

```
# Check residuals for the AR model
checkresiduals(fit_ar)
```

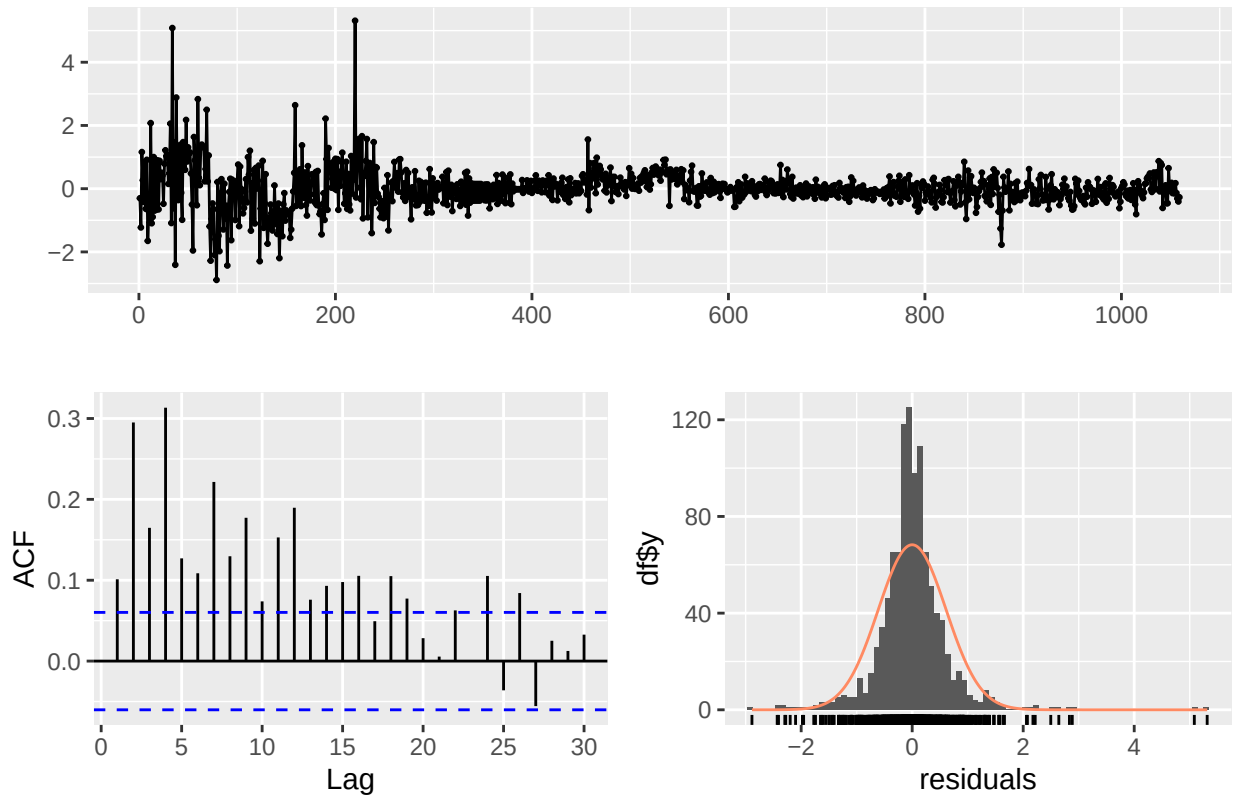
## Residuals from ARIMA(1,0,0) with non-zero mean



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,0,0) with non-zero mean
## Q* = 126.55, df = 9, p-value < 2.2e-16
##
## Model df: 1.   Total lags used: 10
```

```
# Check residuals for the MA model
checkresiduals(fit_ma)
```

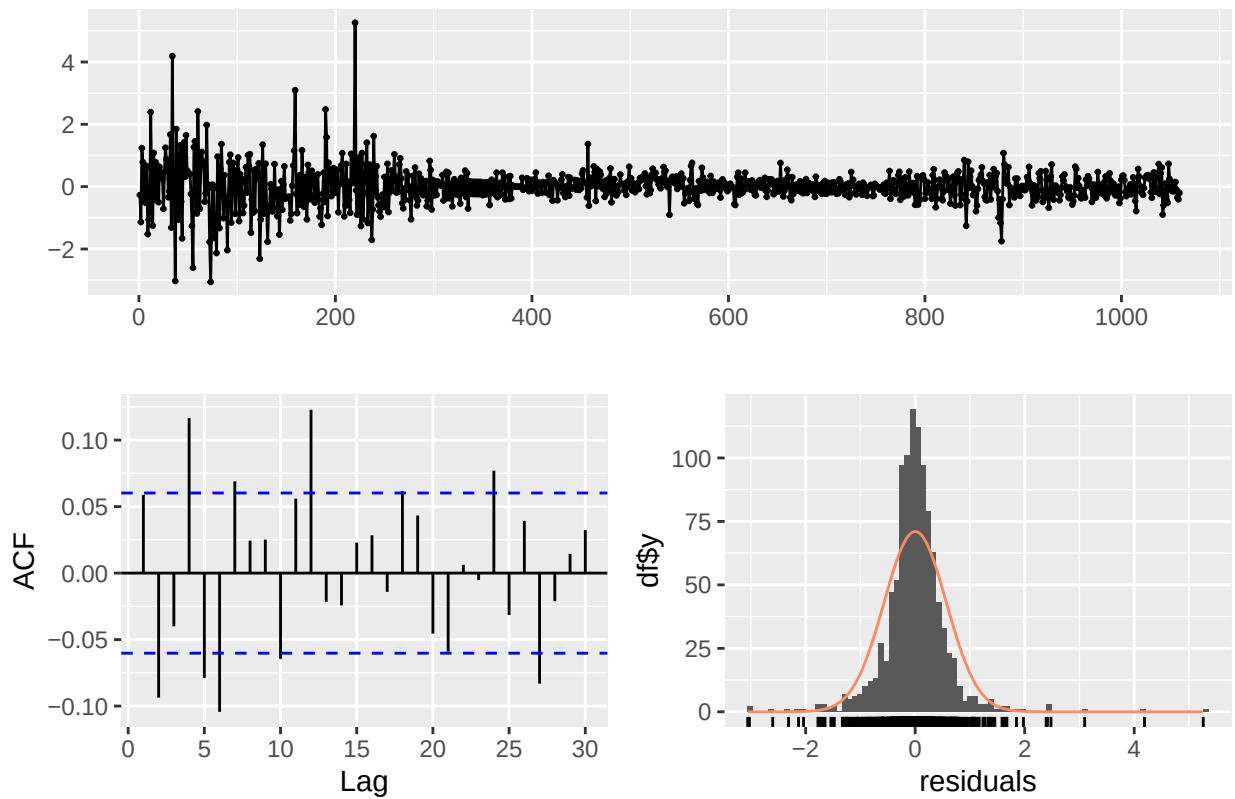
### Residuals from ARIMA(0,0,1) with non-zero mean



```
##  
##  Ljung-Box test  
##  
## data:  Residuals from ARIMA(0,0,1) with non-zero mean  
## Q* = 376.56, df = 9, p-value < 2.2e-16  
##  
## Model df: 1.    Total lags used: 10
```

```
# Check residuals for the ARMA model  
checkresiduals(fit_arma)
```

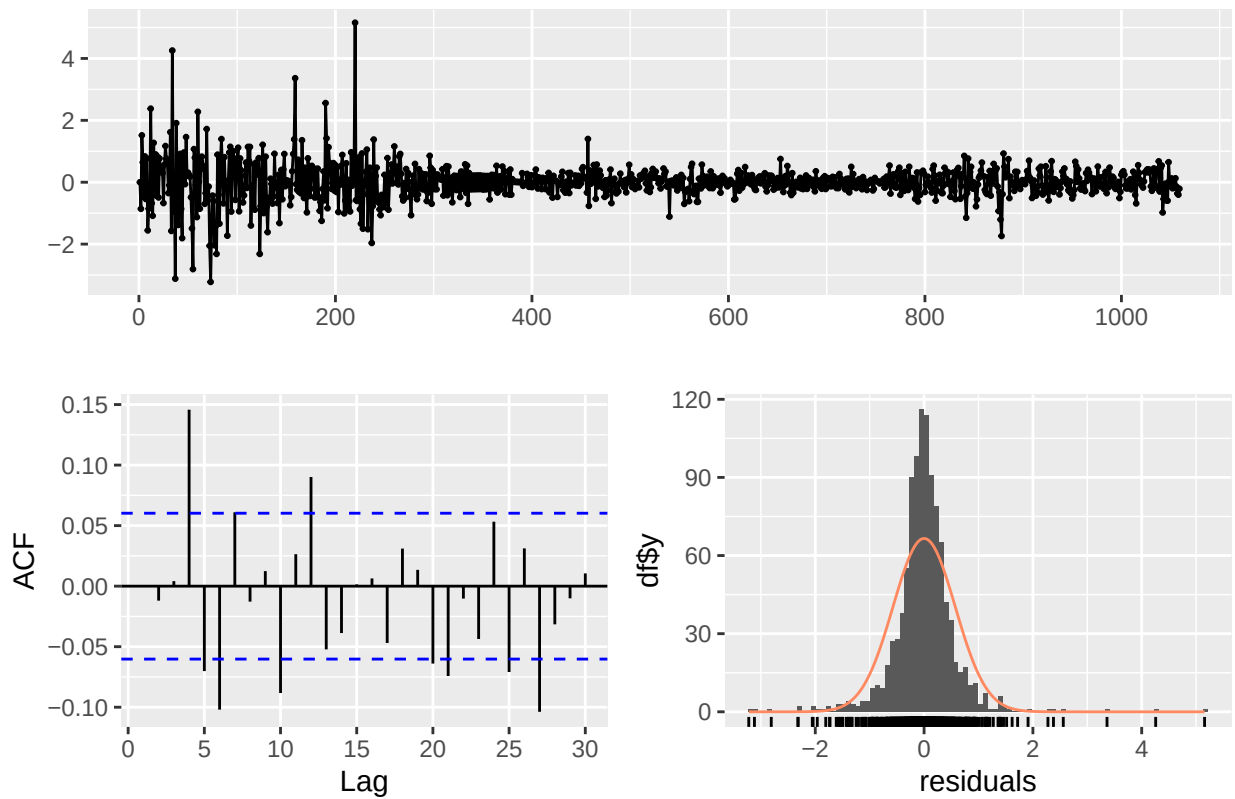
Residuals from ARIMA(1,0,1) with non-zero mean



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,0,1) with non-zero mean
## Q* = 58.228, df = 8, p-value = 1.036e-09
##
## Model df: 2.   Total lags used: 10
```

```
# Check residuals for the SARIMA model
checkresiduals(fit_sarima)
```

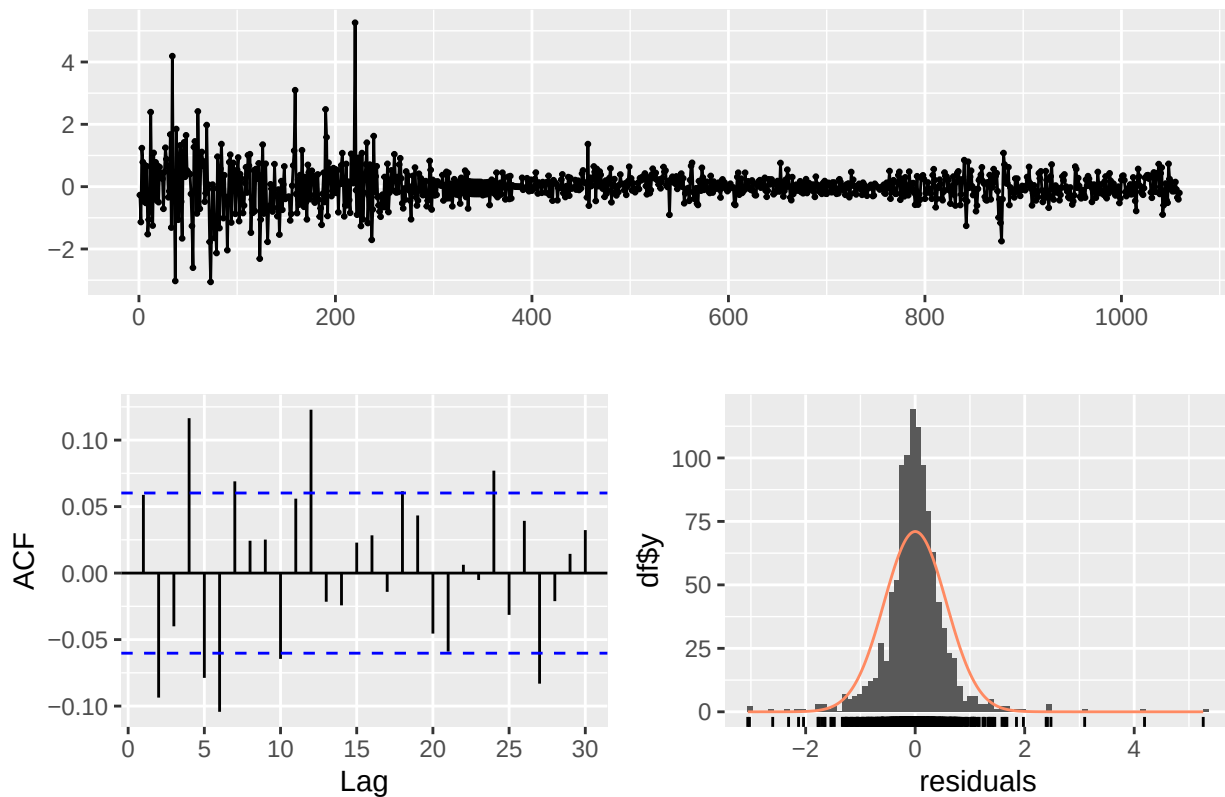
## Residuals from ARIMA(1,1,1)



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1)
## Q* = 51.795, df = 8, p-value = 1.844e-08
##
## Model df: 2.   Total lags used: 10
```

```
# Check residuals for the ARIMA model
checkresiduals(fit_arma)
```

## Residuals from ARIMA(1,0,1) with non-zero mean



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,0,1) with non-zero mean
## Q* = 58.228, df = 8, p-value = 1.036e-09
##
## Model df: 2.   Total lags used: 10
```

## Step 5: Box-Ljung test:

```
# Ljung-Box test for residual autocorrelation
Box.test(fit_ar$residuals, lag = 10, type = "Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  fit_ar$residuals
## X-squared = 126.55, df = 10, p-value < 2.2e-16
```

```
Box.test(fit_ma$residuals, lag = 10, type = "Ljung-Box")
```

```
##
```

```
## Box-Ljung test
##
## data: fit_ma$residuals
## X-squared = 376.56, df = 10, p-value < 2.2e-16
```

```
Box.test(fit_arma$residuals, lag = 10, type = "Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: fit_arma$residuals
## X-squared = 58.228, df = 10, p-value = 7.83e-09
```

```
Box.test(fit_sarima$residuals, lag = 10, type = "Ljung-Box")
```

```
##
## Box-Ljung test
##
## data: fit_sarima$residuals
## X-squared = 51.795, df = 10, p-value = 1.245e-07
```

```
Box.test(fit_arima$residuals, lag = 10, type = "Ljung-Box")
```

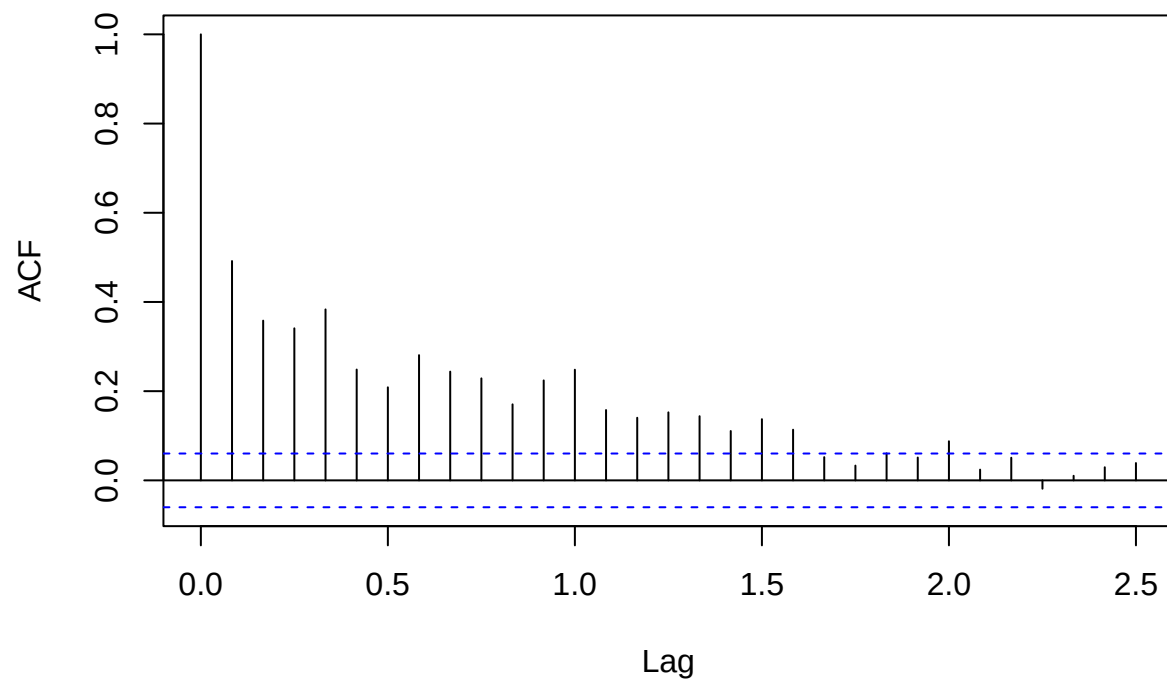
```
##
## Box-Ljung test
##
## data: fit_arima$residuals
## X-squared = 58.228, df = 10, p-value = 7.83e-09
```

## Step 6: ACF AND PACF PLOTS:

```
#convert the original CPIAI data to a time series object
cpiai_ts <- ts(cpi_data$CPIAI, frequency = 12, start = c(1948, 1)) # Adjust start year and frequency i.

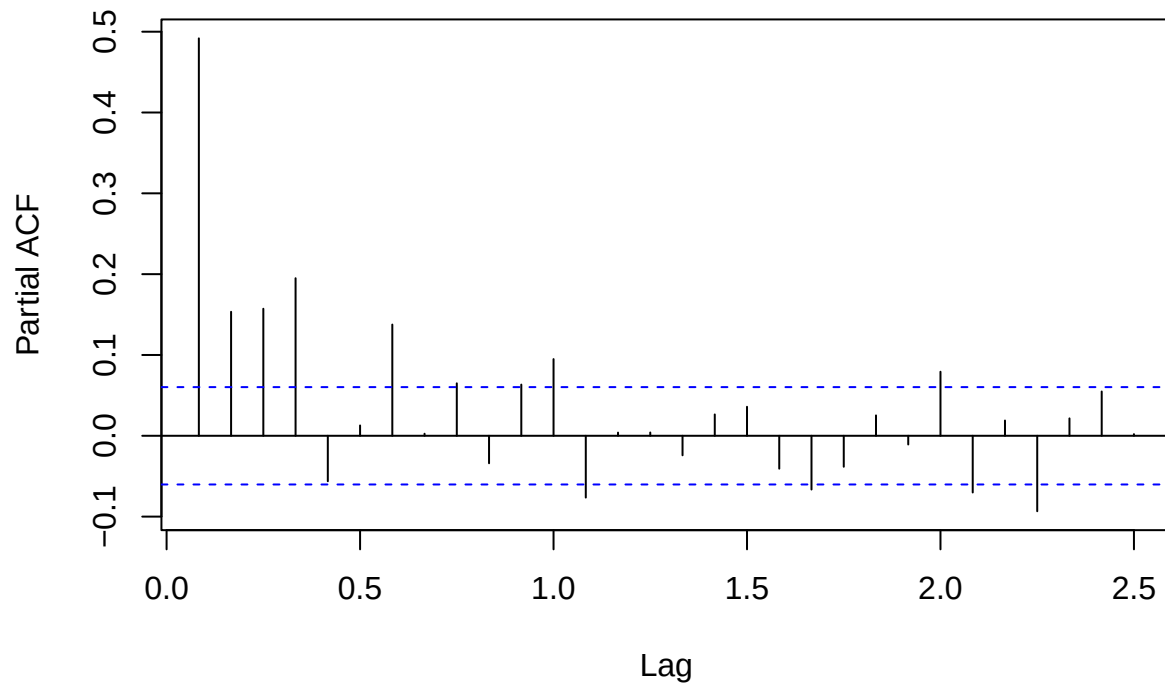
#ACF and PACF for the original data
acf(cpiai_ts, main = "ACF of CPIAI")
```

### ACF of CPIAI



```
pacf(cpiat_ts, main = "PACF of CPIAI")
```

## PACF of CPIAI



### Step 7: Forecasting

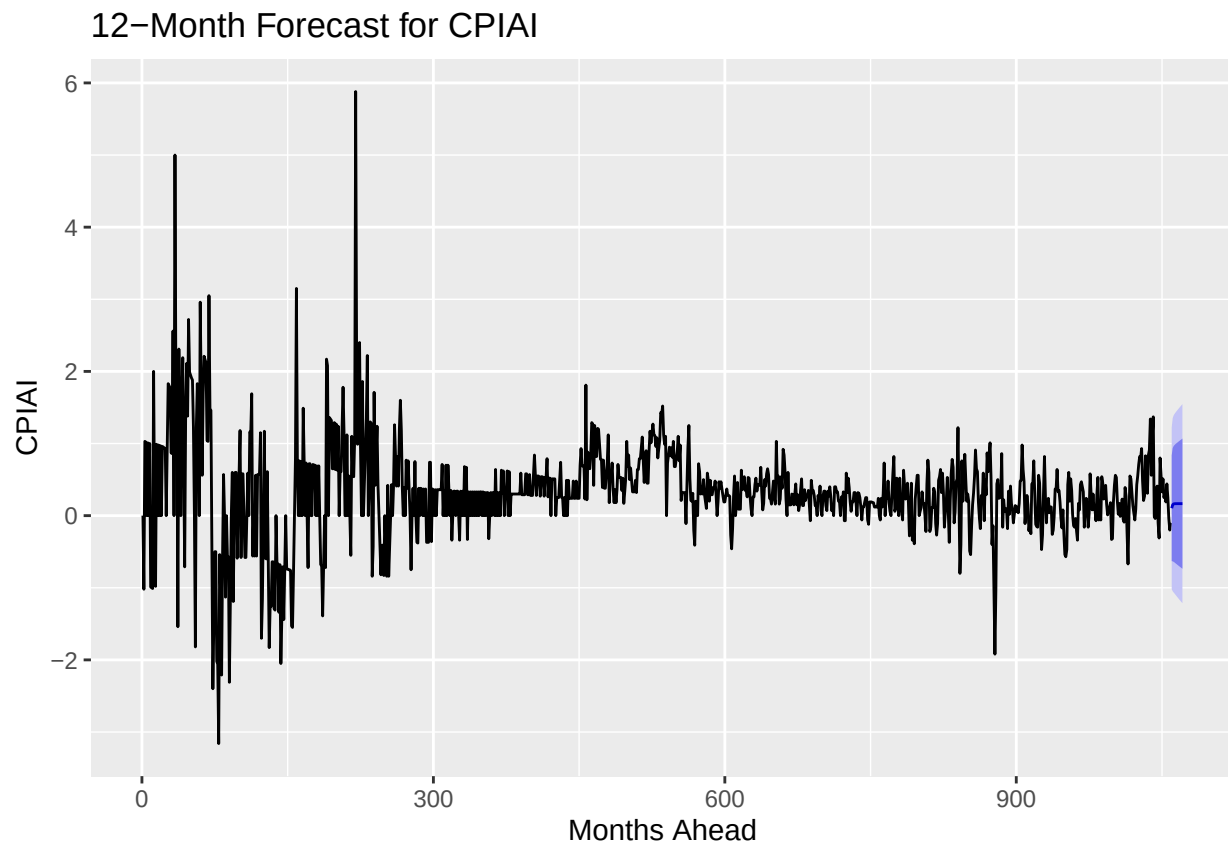
```
# Forecasting the next 12 months
# fitting SARIMA model
fit_sarima <- Arima(cpi_data$CPIAI, order = c(1, 1, 1), seasonal = c(1, 1, 1)) # SARIMA model

# Check the fitted SARIMA model
summary(fit_sarima)
```

```
## Series: cpi_data$CPIAI
## ARIMA(1,1,1)
##
## Coefficients:
##      ar1      ma1
##      0.2382 -0.8586
## s.e.  0.0423  0.0254
##
## sigma^2 = 0.3277: log likelihood = -910.51
## AIC=1827.02  AICc=1827.04  BIC=1841.91
##
## Training set error measures:
##              ME      RMSE      MAE MPE MAPE      MASE      ACF1
## Training set 0.0005467244 0.5716334 0.3622792 NaN  Inf 0.8595134 0.0007668526
```

```
# Forecast using the SARIMA model
cpi_forecast <- forecast(fit_sarima, h = 12)

# Plot the forecast
autoplot(cpi_forecast) +
  ggtitle("12-Month Forecast for CPIAI") +
  xlab("Months Ahead") +
  ylab("CPIAI")
```



## Step 9: Final Conclusion :

In this analysis, I applied various models—AR, MA, ARMA, and SARIMA—to predict CPIAI values. Among these, the **SARIMA(1,1,1)** model, which captures both trend and seasonality, proved to be the most accurate, as evidenced by its strong performance according to AIC and BIC criteria. The Ljung-Box test showed that the residuals were white noise, confirming the model's effectiveness in capturing the underlying data patterns. By forecasting the next 12 months, can anticipate future CPIAI trends, showcasing the power of time series forecasting for economic insights.